

AI Operations (AIOps)

Module 2 Student Handouts

Quick-reference sheets for Lectures 1, 2 & 3 — keep these next to you during labs

Handout 1 — Distributed Data Processing with Apache Spark

Big idea: Spark splits data across a cluster and runs the same computation on every partition in parallel — vertical scaling (a bigger machine) has a ceiling; horizontal scaling (more machines) doesn't.

Spark Architecture

Component	Role
Driver	Runs your program; builds the DAG; coordinates the cluster
Cluster Manager	Allocates resources (standalone, YARN, Kubernetes)
Executors	Worker processes that run tasks and hold data partitions
Tasks	Smallest unit of work — one task per partition

RDDs vs. DataFrames vs. Spark SQL

- RDD — low-level, no schema, rarely needed directly today.
- DataFrame — distributed table with schema; the default choice for ETL.
- Spark SQL — SQL syntax against DataFrames; same optimized engine, same performance.

Lazy Evaluation

- Transformations (filter, select, groupBy, join) — build the plan, don't touch data.
- Actions (count, collect, write, show) — trigger real execution.

Partitioning & Shuffles

- A shuffle = redistributing data across the network (groupBy, join, repartition).
- The most expensive operation in Spark — minimize it where possible.
- Broadcast join: send a SMALL table to every executor instead of shuffling a huge one.

Lab 1 Command / Code Cheat Sheet

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("lab").getOrCreate()

df = spark.read.parquet("data/")
df.explain(True) # plan only, no execution

from pyspark.sql import functions as F
result = df.filter(F.col("x") > 0).groupBy("key").agg(F.count("*"))
result.write.mode("overwrite").partitionBy("key").parquet("out/")
```

```
from pyspark.sql.functions import broadcast  
big.join(broadcast(small), "key") # avoids shuffling `big`
```

Handout 2 — Distributed Compute & Training with Ray

Big idea: Ray distributes ARBITRARY Python code (not just tabular ETL) — a task call returns a future immediately; ray.get() resolves it.

Ray Core Vocabulary

Term	Meaning
Task (@ray.remote on a function)	Stateless unit of work; .remote() returns a future (ObjectRef)
Actor (@ray.remote on a class)	Stateful, long-lived worker process
Object Store	Shared-memory store for large objects, avoids repeated serialization

Data Parallelism vs. Model Parallelism

Strategy	What's Split	When
Data Parallelism	The dataset; full model copy per worker	Model fits on one device; data is huge
Model Parallelism	The model itself, across devices	Model itself doesn't fit on one device (e.g. large LLMs)

Fault Tolerance

- Ray Train supports checkpointing + automatic worker recovery.
- Makes spot/preemptible GPU instances economically viable for training.

Lab 2 Command / Code Cheat Sheet

```
import ray
ray.init()

@ray.remote
def task(x):
    return x * x

futures = [task.remote(i) for i in range(8)]
results = ray.get(futures) # blocks until done

@ray.remote
class Counter:
    def __init__(self): self.n = 0
    def inc(self): self.n += 1; return self.n

c = Counter.remote()
ray.get(c.inc.remote())

from ray.train.torch import TorchTrainer
from ray.train import ScalingConfig
```

```
trainer = TorchTrainer(train_loop_per_worker,  
    scaling_config=ScalingConfig(num_workers=4, use_gpu=False))  
trainer.fit()
```

Handout 3 — Feature Stores (Feast) & Data Quality (Great Expectations)

Big idea: the SAME feature definition must populate both training (offline) and serving (online) — otherwise you get training-serving skew.

Feast Vocabulary

Term	Meaning
Entity	The object a feature describes (e.g. user_id)
Feature View	Named group of features tied to an entity + data source
Offline Store	Historical, point-in-time-correct store — for TRAINING
Online Store	Low-latency key-value store — for SERVING
Feature Service	Named, versioned bundle of feature views a model consumes

Point-in-Time Correctness

Each training row's features must reflect only information available BEFORE that row's label timestamp — otherwise the model trains on "future" information (label leakage). Feast's offline store handles this automatically.

Feast Command / Code Cheat Sheet

```
$ feast apply
$ feast materialize-incremental $(date +%Y-%m-%d)

from feast import FeatureStore
store = FeatureStore(repo_path="feature_repo/")

training_df = store.get_historical_features(
    entity_df=labels_df, features=["user_click_stats:click_count_7d"]
).to_df()

online = store.get_online_features(
    features=["user_click_stats:click_count_7d"],
    entity_rows=[{"user_id": 42}]
).to_dict()
```

Great Expectations Vocabulary

Term	Meaning
Expectation	A single testable assertion (e.g. not_null, mean_between)
Expectation Suite	Named collection of expectations for a dataset
Checkpoint	Repeatable way to run a suite against a batch of data
Data Docs	Auto-generated HTML report of pass/fail results

Great Expectations Command / Code Cheat Sheet

```
import great_expectations as gx
context = gx.get_context()

validator = context.sources.pandas_default.read_parquet("data.parquet")
validator.expect_column_values_to_not_be_null("income")
validator.expect_column_mean_to_be_between("income", min_value=30000, max_value=120000)
validator.save_expectation_suite("my_suite")

checkpoint = context.add_or_update_checkpoint(name="my_checkpoint", validator=validator)
result = checkpoint.run()
print(result.success)
context.open_data_docs()
```

End-of-Module Checklist

1. A Spark ETL pipeline that reads, transforms, and writes partitioned Parquet at scale.
2. A Ray program using at least one task, one actor, and one Ray Train job.
3. A Feast feature repo with a working offline + online retrieval example.
4. A Great Expectations suite that correctly FAILS on a dataset with known issues.